

Question 9

Memory utilization factor shall be computed as

Options:

- (a) Memory in use / Allocated memory
- (b) Memory in allocated memory / Total memory available
- (c) Memory in use / Total memory available
- (d) None of these

 **Correct Answer: (c) Memory in use / Total memory available**

Explanation

The **memory utilization factor** tells us **how efficiently the system is using the available memory**.

To measure this, we compare:

- **How much memory is actually being used**
vs.
- **How much total memory is available in the system**

So the formula becomes:

$$\text{Memory Utilization} = \frac{\text{Memory in Use}}{\text{Total Memory Available}}$$

Example

Suppose a computer has:

- **Total memory = 8 GB**
- **Memory being used = 6 GB**

Then:

$$\text{Memory Utilization} = \frac{6}{8} = 0.75 = 75\%$$

This means the system is using **75%** of its memory.

Why Other Options Are Wrong

Option	Why Incorrect
(a) Memory in use / Allocated memory	Allocated memory is only part of total memory, not the whole system.
(b) Memory allocated / Total memory available	It shows allocation ratio, not actual utilization.
(d) None of these	Not correct because option (c) is correct.

Final Answer: (c) Memory in use / Total memory available 

Question 20

An operating system contains 3 user processes, each requiring 2 units of resource R.
We need to find the minimum number of units of R such that no deadlocks will ever arise.

Options:

- (a) 4
 - (b) 3
 - (c) 5
 - (d) 6
-

Understanding the Problem

- Number of processes = 3
- Each process needs a maximum of 2 units of resource R.

To avoid deadlock, the system must ensure that at least one process can get all the resources it needs, finish, and release them.

Worst-case scenario

Each process holds 1 unit and waits for 1 more unit to complete.

So, in that situation:

- Allocated units = 3 (one to each process)
- We must have at least 1 extra unit to give to one process to allow it to finish.

So:

$$\text{Minimum resource units} = \text{Processes} + 1 = 3 + 1 = 4$$

✔ Correct Answer: (a) 4

Explanation in One Line

With 4 units, even if each process holds 1 unit, there is 1 free unit to complete one process and avoid deadlock.

Question 50

Q: Average process size = **s** bytes.

Each page entry = **e** bytes.

The **optimum page size** is given by:

Options:

- (a) $\sqrt{s}$
- (b) $\sqrt{2se}$
- (c) s
- (d) $\sqrt{se}$

✔ Correct Answer: (b) $\sqrt{2se}$

Explanation

The **optimal page size (P)** is determined to reduce fragmentation and page table size.

$$P = \sqrt{2 \cdot s \cdot e}$$

Where:

- **s** = average process size
- **e** = size of each page table entry

So correct formula is **$\sqrt{2se}$** .

Question 51

Given:

The **aging algorithm** with $a = \frac{1}{2}$ is used to predict next run time.

The **previous four run times** (from oldest to newest):

40, 20, 10, 15 ms.

Formula (Exponential Averaging):

$$T_{n+1} = a \cdot T_n + (1 - a) \cdot T_n$$

But with aging, we apply weights like:

$$T_{pred} = a^3 \cdot T_{oldest} + a^2 \cdot T_{next} + a^1 \cdot T_{next} + a^0 \cdot T_{latest}$$

Here $a = \frac{1}{2}$

So:

$$T_{pred} = \left(\frac{1}{2}\right)^3(40) + \left(\frac{1}{2}\right)^2(20) + \left(\frac{1}{2}\right)^1(10) + \left(\frac{1}{2}\right)^0(15)$$

Calculate:

Term	Value
------	-------

$(1/2)^3 \times 40$	5
---------------------	---

$(1/2)^2 \times 20$	5
---------------------	---

$(1/2)^1 \times 10$	5
---------------------	---

$(1/2)^0 \times 15$	15
---------------------	----

$$T_{pred} = 5 + 5 + 5 + 15 = 30 \text{ ms}$$

✔ Correct Answer: 30 ms

Final Answers Summary

Question Correct Answer Explanation

49	(b) 3	Process has 3 classical states: Ready, Running, Blocked
50	(b) $\sqrt{2se}$	Formula for optimum page size
51	30 ms	Calculated using aging algorithm

Question 59: Page Faults with FIFO

The question asks for the total number of **page faults** that will occur using the **First-In, First-Out (FIFO)** page replacement algorithm under specific conditions.

The Problem Details

- **Virtual Pages:** 5 pages, numbered 0 to 4.
- **Page Frames (Memory Slots):** 3 frames.
- **Reference String (Order of access):** 0, 1, 2, 3, 0, 1, 4, 0, 1, 2, 3, 4.

Applying the FIFO Algorithm

The **FIFO** algorithm works by replacing the page that has been in memory for the longest time (the "oldest" page) when a new page needs to be loaded and all page frames are full.

Here is the step-by-step simulation:

Reference	Frame 1	Frame 2	Frame 3	Page Fault?	Notes
0	0			YES	Frame 1 gets 0 (oldest is 0).
1	0	1		YES	Frame 2 gets 1 (oldest is 0).
2	0	1	2	YES	Frame 3 gets 2 (oldest is 0).
3	3	1	2	YES	0 is oldest, so 3 replaces 0.
0	3	0	2	YES	1 is oldest, so 0 replaces 1.
1	3	0	1	YES	2 is oldest, so 1 replaces 2.
4	4	0	1	YES	3 is oldest, so 4 replaces 3.
0	4	0	1	NO	0 is already in memory.
1	4	0	1	NO	1 is already in memory.
2	4	0	2	YES	1 is oldest, so 2 replaces 1.
3	3	0	2	YES	4 is oldest, so 3 replaces 4.
4	3	4	2	YES	0 is oldest, so 4 replaces 0.

Result

Counting the number of YES in the "Page Fault?" column:

$$3 + 3 + 3 = \mathbf{9}$$

The total number of page faults is **9**.

Conclusion

The correct option is **(d) 9**.

61.

Reasoning (simple worst-case argument):

- Let $m = 6$ tape drives and each process may need at most $k = 2$ drives.
- In the worst case for deadlock, each process could already hold $k - 1$ drives and be waiting for one more to finish.
Here $k - 1 = 1$, so each process might hold 1 drive.

- If there are n processes, those held drives total $n \cdot (k - 1) = n \text{ drives}$. To guarantee progress we must have **at least one free drive** so one process can get its final drive, finish, and release resources.
- Thus we need $m \geq n \cdot (k - 1) + 1$. Rearranging for n :

$$n \leq \frac{m - 1}{k - 1}.$$

- Plugging $m = 6, k = 2$:

$$n \leq \frac{6 - 1}{1} = 5.$$

So the largest integer n that guarantees no deadlock is **5**.

(If $n = 6$ you can construct a deadlock: give 1 drive to each of the 6 processes — no free drives remain, and each needs one more $\rightarrow$ circular wait.)

•

Question 71 asks about which storage placement strategy places a program in the **largest available hole** (free space) in memory.

The correct strategy is the **Worst Fit** algorithm.

71.

Explanation of Worst Fit

The **Worst Fit** memory allocation strategy works as follows:

1. When a process requests a block of memory, the operating system searches the entire list of free memory partitions (holes).
2. It selects the **largest free partition** (the "worst fit") that can accommodate the process's size.
3. The process is loaded into this large hole.
4. The remaining, smaller piece of the large hole is added back to the list of free partitions.

Why it's used:

The goal of Worst Fit is to leave the **largest possible free chunk of memory remaining** after an allocation, hoping that this large remaining chunk will be useful for future, larger memory requests. However, it often leads to a high degree of **external fragmentation**, as it tends to break down large free blocks into many small, unusable pieces over time.

Options Review:

- **(a) Best fit:** Places the program in the **smallest** available hole that is large enough.
- **(b) First fit:** Places the program in the **first** available hole that is large enough, starting from the beginning of the free list.

- **(c) Worst fit:** Places the program in the **largest** available hole. (This is the correct answer).
- **(d) Buddy:** A memory allocation technique that divides memory into partitions that are powers of 2.

The correct choice is **(c) Worst fit**.

Question 93: Logical Address Size in Paging

Question 93 asks for the number of bits required for the **logical address** in a simple paging system, given the size of the logical address space and the page size.

Problem Details

- **Logical Address Space Size:** 2^{24} bytes
- **Page Size:** 2^{10} bytes

The Formula

In a paged system, the Logical Address is divided into two parts:

1. **Page Number (p):** Used to index the Page Table.
2. **Page Offset (d):** Used to locate a byte within the page.

$$\text{Size of Logical Address (bits)} = \text{Number of bits for Logical Address Space}$$

The number of bits for the logical address space is found by taking the $\log_2$ of its size.

$$\text{Logical Address Size (bits)} = \log_2(\text{Logical Address Space Size})$$

Calculation

Given the Logical Address Space Size is 2^{24} bytes:

$$\text{Logical Address Size (bits)} = \log_2(2^{24})$$

$$\text{Logical Address Size (bits)} = 24$$

The total number of bits in the logical address is **24**.

Alternative Check (Page and Offset Bits)

1. **Page Offset (d) bits:** $\log_2(\text{Page Size}) = \log_2(2^{10}) = 10$ bits.
2. **Page Number (p) bits:** $\log_2\left(\frac{\text{Logical Address Space}}{\text{Page Size}}\right) = \log_2\left(\frac{2^{24}}{2^{10}}\right) = \log_2(2^{24-10}) = \log_2(2^{14}) = 14$ bits.
3. **Total Logical Address bits:** $p + d = 14 + 10 = \mathbf{24}$ bits.

The total number of bits in the logical address is **24**. Since 24 is not an option, there must be a typo in the question's options. Let's re-examine the options: (a) 4, (b) 14, (c) 10, (d) 18.

- **10** is the page offset size.
- **14** is the page number size.

- **24** is the total logical address size.

Assumption: Given the available options, and the common practice of UGC-NET to sometimes make a mistake, it is highly probable that the question intended to ask for either the **Page Number bits** or the total Logical Address size where the options are simply wrong. Since **14** is an option, it is the most likely intended answer, representing the **page number bits**. However, the question *explicitly* asks for the total bits in the **logical address**, which is **24**.

Based on the explicit question: 24 bits. (Not an option)

Based on the only plausible option (b): 14 bits (This is the page number size).

In the context of typical exam errors, **(b) 14** is the most probable answer if the question *meant* to ask for the page number bits.

Question 94: Deadlock in Resource Sharing

Question 94 asks about the possibility of a **deadlock** occurring in a system with shared resources.

Problem Details

- **Processes (\$n\$):** 3
- **Shared Resources (\$R\$):** 4 units
- **Maximum Need per Process (\$M\$):** 2 units

Deadlock Condition (Avoidance Check)

A system is guaranteed to be deadlock-free if the total resources (\$R\$) satisfy the condition:

$$R \geq n \times (M - 1) + 1$$

Where:

- $n = 3$ (processes)
- $R = 4$ (resources)
- $M = 2$ (max need)

Let's plug in the values to check for the safe state guarantee:

$$4 \geq 3 \times (2 - 1) + 1$$

$$4 \geq 3 \times (1) + 1$$

$$4 \geq 3 + 1$$

$$4 \geq 4$$

Since the condition **$4 \geq 4$ is true**, the system is guaranteed to be in a **safe state**, and therefore, **deadlock can never occur**.

Worst-Case Scenario Check

Let's verify the worst-case allocation (the point just before a guaranteed safe state):

1. Each of the 3 processes holds $M - 1 = 2 - 1 = 1$ resource unit.
2. **Total allocated:** $3 \times 1 = 3$ units.
3. **Resources remaining (Available):** $4 - 3 = 1$ unit.
4. This 1 available unit is enough to satisfy the remaining need (1 unit) of any one of the 3 processes. That process completes, releases its 2 units, and the system becomes safe.

Since the system can always progress and a deadlock state is impossible, the correct choice is **(a) can never occur**.

The correct answer for Q94 is **(a) can never occur**.

98. Calculate Page Faults with FIFO

The problem requires determining the number of page faults generated by the given address sequence in a pure demand paging system using the FIFO replacement algorithm, with 100 records per page and 1 free main memory frame.

The system parameters are:

- **Records per page:** 100
- **Number of Page Frames (Memory):** 1 (1 free frame)
- **Page Replacement Algorithm:** FIFO (First-In, First-Out)
- **Address Sequence (Decimal):** 0100, 0200, 0430, 0499, 0510, 0530, 0560, 0120, 0220, 0240, 0260, 0320, 0370.

The first step is to convert the logical addresses into their corresponding Page Numbers.

Solution: The total number of page faults is 7.

This corresponds to option (c).